Independent Validation of Generated Test Cases

Response to the research panel comment, the mechanism built, the evidence, and how accuracy improves with different models

NextGen QA (R26-SE-039) · Component 2 — Intelligent Test Case Generation

Abeygunasekara D T · IT22303684 · SLIIT Faculty of Computing

Supervisor: Ms. Suriyaa Kumari · Co-supervisor: Mr. Eishan Weerasinghe

13 October 2026

Contents

- 1. The panel comment and the one-paragraph answer
- 2. Why the previous pipeline was not independent
- 3. The mechanism, layer by layer
- 4. Scoring and verdicts
- 5. The auto-fix loop
- 6. What the panel can see in the dashboard
- 7. Evidence on the live system
- 8. Experiment: does a different model give more accurate results?
- 9. How to make it more accurate, and what it costs
- 10. Likely viva questions and answers
- 11. Limitations stated honestly
- Appendix: configuration, endpoints, reproduction

1. The panel comment and the answer

> *Panel comment (progress presentation): “Make sure to have a mechanism to independently validate the generated test cases.”*

Answer. Component 2 now has a validation gate that runs on every generated artefact — the Gherkin scenarios and the executable test suites — before approval and before execution. It combines two signals that do not depend on the generator: deterministic static analysis that needs no model at all, and an LLM judge that runs as a separate reviewer on a different model from the one that wrote the artefact. The gate produces a predicted pass rate per scenario, concrete findings with line numbers and suggested fixes, an explainable score, and a reviewable auto-fix that is applied only as a new version with the original kept. The predictions are then compared against the real CI/CD execution results, so the validator itself is measured. Everything is persisted, so each judgement is auditable: which model judged, what it found, what was fixed, and what the run showed.

Where it lives. Backend package `backend/app/validation/` (static checks, judge, fixer, scoring service, API), table `validation_reports`, and the Validation panel in the Gherkin editor, Code Review and Suite editor pages. Documentation: `VALIDATION.md`; tests: `tests/test_validation_static.py` (21 unit tests, no LLM, no database).

2. Why the previous pipeline was not independent

Before this work, a generated test was accepted on two signals only: the LLM that produced it (which cannot be trusted to grade its own output) and the QA engineer who clicked Approve (which does not scale and gives no written reason). The LLM-as-a-judge literature (Zheng et al., 2023) documents self-enhancement bias: a model prefers its own output. Independence therefore has to come from three places: checks that use no model, a judge that is a different model with a fixed rubric, and ground truth from real execution.

Layer	Independence comes from	Needs an LLM?
Deterministic static checks	Pure functions of the artefact text, the crawled DOM map and the source Gherkin	No
LLM judge	A different model (VALIDATOR_MODEL) in a reviewer role with a different prompt, told not to trust the code	Yes, a second one
Execution calibration	The real CI/CD result of the same suite, compared with the prediction	No
Human review	QA sees findings, per-scenario prediction and the diff before approving	No

3. The mechanism, layer by layer

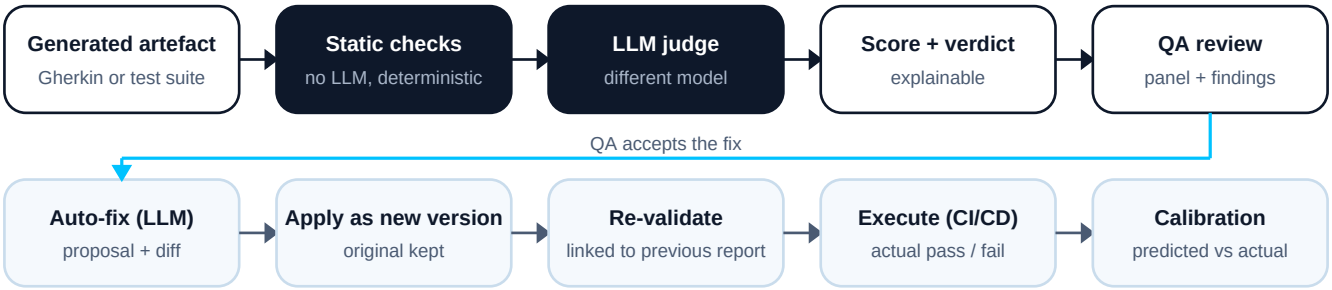

Figure 1. Validation gate (top) and the fix-and-measure loop (bottom).

3.1 Deterministic static checks (test suites)

Check	Severity	Why it matters
Python syntax (ast.parse); JavaScript bracket/string scanner + node --check	critical	The file will not be collected: every test fails
No test functions found	critical	The runner collects zero tests
Unresolved <> locator in DOM-aware mode	critical	Not a valid selector (info only in abstract mode)
Test with no assertion (direct, expect, .should, explicit wait, or via a helper that asserts)	major	Vacuous: it can never fail on a defect
Empty test body; test calls another test	major	Trivial pass; fixtures not injected -> TypeError
Gherkin scenario with no implementing test	major	A requirement silently dropped
Headed browser forced; known fatal anti-pattern (sanitizer dry-run)	major	Crashes on CI; hangs until timeout
Selectors not in the crawled element map (one grouped finding, nearest candidates listed)	minor	Unverified: uncrawled page or invented locator
Long fixed sleeps, leftover TODOs	minor	Flakiness, incompleteness

3.2 Deterministic static checks (Gherkin)

Feature header present; at least one scenario; every scenario has a *When* (action) and a *Then* (observable outcome); outlines have Examples with matching columns; no duplicate titles; no vague wording (“works correctly”); and every acceptance criterion of the user story maps lexically to at least one scenario (traceability).

3.3 The LLM judge

The judge receives the line-numbered artefact, the source Gherkin, the crawled DOM map explicitly marked as *partial* (a selector missing from the map is unverified, not wrong), known facts about the target application, and the static findings already found so it does not repeat them. It must answer in strict JSON: a holistic pass probability, per-scenario *pass / fail / uncertain* with confidence and a one-line reason, findings with severity, category, line, snippet, suggestion and a fixable flag, and strengths. The prompt defines the severity scale (critical: file will not run or one root cause fails most tests; major: one test fails or is vacuous; minor: fragile or unverified; info: observation). For Gherkin it also returns a semantic acceptance-criteria coverage table.

Independence knob: `VALIDATOR_PROVIDER / VALIDATOR_MODEL / VALIDATOR_API_KEY` override the generator's settings. In the evaluated configuration the generator is gpt-4o-mini and the judge is

gpt-4o; the report records the judging model and flags whether it differed from the generator.

4. Scoring and verdicts

Term	Definition
static_score	100 - penalties (critical 25, major 10, minor 3), floored at 0
llm_score	judge's holistic pass probability, 0–100
scenario_rate	scenarios predicted to pass ÷ scenarios judged × 100
base	0.5 × llm_score + 0.3 × scenario_rate + 0.2 × static_score (static only when no judge is available)
caps	any static critical -> at most 15; any judge critical -> at most 40; any major -> at most 85
verdict	likely_failing if any critical or score < 50 · ready if score >= 80 and no major · otherwise needs_review

The breakdown (weights, caps applied, counts, judge model, independence flag) is stored with every report and shown in the dashboard under “How the score is computed”, so the number is defensible rather than a black box. Every report also stores the SHA-256 of the text it validated; when the text changes, the UI marks the report as outdated.

5. The auto-fix loop

The fixer receives the artefact plus the selected findings (ids, lines, suggestions), the Gherkin and the partial DOM map, and must return the minimally changed artefact between explicit markers followed by a change log that maps each edit to a finding id. The deterministic sanitizer runs on the result, and the static checks run again to report a before/after deterministic score. Nothing is saved automatically: the dashboard shows a side-by-side diff, the change log and the remaining findings. Applying creates a **new suite version** (the original is kept) and triggers re-validation linked to the previous report through `parent_report_id`, so the score movement is recorded. Findings the fixer cannot resolve with the information it has (for example a TODO that needs an uncrawled selector) are marked “manual fix” and excluded.

6. What the panel can see in the dashboard

- **Code Review**: each suite card shows its predicted pass percentage, verdict (Ready / Needs review / Likely failing), predicted-pass scenarios and critical/major counts; “Validate All Suites”; a summary strip with validated count, average score and predicted-versus-actual calibration.

- **Suite editor**: a Validate button and a right-hand panel with the score ring, per-scenario prediction with reasons, findings grouped by severity and tagged *Static check* or *AI judge*, clickable line numbers that jump into the code editor, and the score breakdown.

- **Auto-fix with AI**: diff dialog with the change log and the deterministic before/after score; “Apply as new version & re-validate” shows the score moving from X to Y.

- **Gherkin editor**: the same panel and fix flow for scenarios, plus acceptance-criteria coverage.

- **API** (Swagger): POST/GET /api/v1/validation/suites/{id}, …/fix, the Gherkin equivalents, /validation/projects/{id}/latest and /summary (calibration), /validation/reports/{id}.

7. Evidence on the live system

Measured on the SauceDemo project in the production database on 10 October 2026 (generator gpt-4o-mini, judge gpt-4o).

Artefact	Static	Judge	Scenarios predicted pass	Score / verdict	Key findings
Playwright suite v28 (91 lines)	87	70	3 / 8	40 · likely failing	One Gherkin scenario had no implementing test (Gherkin edited after generation); 6 selectors unverified by the inventory-page crawl; checkout step contradicts its scenario
Selenium suite v2	90	—	—	static only	5 unverified selectors; 2 leftover TODOs; no vacuous tests after helper-aware assertion detection
Cypress suite v3	97	—	—	static only	7 unverified selectors
Gherkin “Remove a product from the cart”	100	85	1 / 2	78 · needs review	Ambiguous outcome for the empty-cart scenario; both acceptance criteria covered

Auto-fix on the Playwright suite produced four described changes (implemented the missing scenario, kept the verified cart-badge selector, aligned the checkout test, added the test function); after the fix the only deterministic finding left was the unverified-selector note. The original version was kept and the new version was re-validated with a link to the previous report.

A quality problem that was found and fixed during development: with gpt-4o-mini as judge and the crawled map presented as “the elements that exist”, the judge declared the login selectors non-existent (the crawl had only covered the inventory page) and the fixer replaced a correct selector with an unrelated one. Rewording the prompts to state that the map is partial, adding verified application facts and a severity scale corrected this. It is a reproducible example of why evidence framing and model choice matter — which leads to the experiment below.

8. Experiment: does a different model give more accurate results?

Design. The active Playwright suite (v28, 8 tests) was executed locally and all 8 tests passed. The same suite, Gherkin, DOM map and static findings were then given to four judge models, two runs each, with no other change. Each model's per-scenario prediction was compared with the real outcome of the test it names. Seven of the eight Gherkin scenarios have an implementing test (the eighth is the dropped scenario the static check reports), so accuracy is out of 7. Run on 13 October 2026.

Judge model	Run	Score	Pass prob.	Predicted pass rate	Per-scenario accuracy	Uncertain	Findings	Time
gpt-4o-mini (generator model)	1	59	60	38%	43% (3/7)	0	4	18.1 s
	2	40	40	25%	29% (2/7)	0	4	15.0 s
gpt-4o (current validator)	1	82	85	75%	86% (6/7)	0	3	10.7 s
	2	82	85	75%	86% (6/7)	0	3	16.4 s
gpt-4.1-mini	1	71	70	62%	71% (5/7)	0	5	16.8 s
	2	75	70	75%	86% (6/7)	0	4	17.3 s
gpt-4.1	1	75	85	50%	57% (4/7)	2	6	19.3 s
	2	82	85	75%	86% (6/7)	0	6	15.5 s
Majority vote of gpt-4o, gpt-4.1-mini, gpt-4.1 (6 votes per scenario)	—	—	—	86%	86% (6/7)	—	—	approx. 6 calls

Reading the results.

- The generator's own model (gpt-4o-mini) is the worst judge: it predicted failure for 4–5 of 7 passing tests, driven by the unverified-selector framing, and it was unstable between runs (59 -> 40). This is exactly the case for not letting the generator grade itself.

- gpt-4o was both the most accurate (6/7) and perfectly stable across runs (identical score, probability and findings). gpt-4.1 reached the same accuracy on its second run but hedged with two “uncertain” answers on the first; gpt-4.1-mini sat in between.

- The one test every strong model marked as *fail* — `test_successful_checkout_with_all_details` — passed in execution. On inspection the judge was right and the execution result was misleading: the generated Gherkin for that scenario says *“And I leave the postal code empty”* and still expects a successful order, which contradicts the application; the code generator silently filled a postal code, so the test passes but does not test what the scenario says. The judge found a specification defect that pass/fail could never reveal. Excluding that scenario, the strong models were 6/6.

- A majority vote across the three strong models matched the best single model and removed single-run variance. The cost is three to six judge calls instead of one (about 10–20 seconds each).

9. How to make it more accurate, and what it costs

Improvement	What it does	Expected effect	Cost / status
Use a stronger judge than the generator	VALIDATOR_MODEL=gpt-4o (or Claude Sonnet via VALIDATOR_PROVIDER)	Accuracy 29–43% -> 86%; stable across runs	Done; one call, ~11 s
Ensemble by majority vote	Run 3 judges (or 3 runs) and vote per scenario; treat ties as uncertain	Removes single-run variance (gpt-4.1 run 1 -> run 2)	3–6 calls; add a VALIDATOR_RUNS knob — planned
Self-consistency on one model	Repeat the judge N times, report mean pass probability and spread	Confidence interval instead of a point estimate	N calls; planned
Complete the DOM crawl	Crawl login, inventory, cart and checkout pages, not one page	Removes the ‘unverified selector’ false alarms that misled the small model	No LLM cost; crawler already supports it
Gherkin consistency check	Flag scenarios whose preconditions equal another scenario's but whose outcomes differ (the postal-code case)	Catches specification defects before code is generated	Static rule + judge prompt; planned
Calibration from executions	Keep comparing predicted and actual pass rates (summary endpoint, MAE) and tune weights/caps	Score that tracks reality; evidence for the paper	Implemented; needs more executed suites
Abstain instead of guess	Count ‘uncertain’ as abstain and show it; do not force a pass/fail	Fewer confident wrong answers	Done in the UI
Fine-tune or few-shot from history	Use validated-and-executed pairs as examples in the prompt	Project-specific accuracy	Later; needs data volume

Recommendation for the viva and the paper: keep gpt-4o (or an equivalent strong model) as the independent judge, add the majority-vote option for the final evaluation runs, complete the crawl so grounding findings are precise, and report calibration (predicted versus actual) over all executed suites with mean ± spread over repeated runs.

10. Likely viva questions and answers

Question	Answer
How is this independent if an LLM wrote the code and an LLM judges it?	Three ways: the static layer uses no model and is unit-tested; the judge is a different model with a different role and rubric, and the report records which model judged; and the prediction is compared with the real execution result, so the judge itself is measured. Section 8 shows the generator's own model is a poor judge and a different model is a good one.
Why not just run the tests? Pass/fail is the truth.	Running is part of the gate (calibration), but pass/fail is not enough: a test can pass while testing the wrong thing. The checkout case in Section 8 passed and was wrong; only the judge caught the specification defect. Static checks also catch vacuous tests that pass by construction.
How accurate is the judge?	On the executed suite: 86% per-scenario agreement with gpt-4o, stable across runs; the single disagreement was a real specification defect. The generator's model scored 29–43%. Calibration against more executed suites is the ongoing measurement.
Does the auto-fix change code behind the QA's back?	No. The fix is a proposal shown as a diff with a change log; applying creates a new version, the original is kept, and the new version is re-validated with a link to the previous report.
What if the LLM is unavailable or the key is missing?	The gate still runs: static checks produce a score and the report states that the judge was unavailable.

Question	Answer
How do you avoid the judge hallucinating problems?	It receives the static findings so it does not repeat them, the DOM map is marked partial, verified application facts are supplied, severities are defined, findings must cite line numbers, and uncertain answers are allowed and shown.
Is the score a black box?	No: the formula, weights, caps applied and counts are stored with each report and shown in the UI.
What would you do with more time?	Majority-vote ensemble, full multi-page crawl, a Gherkin consistency rule, and calibration over many executed suites with repeated runs; then report precision/recall of findings against executions in the paper.

11. Limitations stated honestly

- Single benchmark application (SauceDemo) and a handful of stored suites: the mechanism is demonstrated; statistical claims need more executed suites.

- LLM judgements vary between runs on some models; repeated runs and voting are the mitigation.

- Scenario-coverage and acceptance-criteria checks are lexical and can miss paraphrased titles; the judge covers the semantic side.

- The crawl is single-level, so ‘unverified selector’ findings remain until all pages are crawled.

- Screenshots and code are sent to an external provider unless a local model (Ollama) is configured.

Appendix: configuration, endpoints, reproduction

```
VALIDATOR_PROVIDER=openai · VALIDATOR_MODEL=gpt-4o · VALIDATOR_API_KEY=… (each optional; defaults to the generator's LLM_* settings)
```

Method	Path	Purpose
POST	/api/v1/validation/suites/{suite_id}	Validate a suite (body: parent_report_id?, use_llm?)
GET	/api/v1/validation/suites/{suite_id}	Latest report (+ is_current)
POST	/api/v1/validation/suites/{suite_id}/fix	Fix proposal (finding_ids?, code?) — never saved
POST / GET / POST	/api/v1/validation/gherkin/{gherkin_id}[/fix]	Same for Gherkin
GET	/api/v1/validation/projects/{id}/latest?target_type=	Latest report per artefact
GET	/api/v1/validation/projects/{id}/summary	Counts, averages, predicted-vs-actual calibration
GET	/api/v1/validation/reports/{report_id}	One report

Reproduce the experiment: with the backend virtualenv, set the judge model per run and call `judge_suite` on the executed suite, matching predictions to `test_runs` rows of its latest execution; the comparison script and its JSON output are kept alongside this document. Unit tests: `python -m pytest tests` (37 pass).